PYTHON PER NERDS

VIDEOGIOCHI, SIMULAZIONI E GRAFICA ASCII NEL TERMINALE



FRANCESCO CLEMENTONI

Introduzione

Amo Python, ma allo stesso tempo mi suscita sentimenti contrastanti, come in una qualunque «relazione complicata» (cit. Facebook).

E' infatti un linguaggio che nasconde numerose trappole qua e là, proprio come un dungeon di Skyrim, ma senza la possibilità di prendere a mazzate i cattivi!

Ad esempio, dichiari una variabile e le assegni un valore, poi scrivi una funzione, chiamiamola `fus_ro_dah()`, e dentro questa funzione leggi la variabile: tutto a posto, la puoi leggere, è tua.

Ma se nella stessa funzione provi a *modificarla*, ti becchi un bel *UnboundLocalError*!

MA CHE COMPORTAZIONE È MAI QUESTA? (cit)

```
cattivo = 'Nord'

def fus_ro_dah():
    print(f"Ora faccio volare {cattivo}")
    cattivo = 'Elfo dei boschi'  # ERRORE!!! UnboundLocalError!
```

Nonostante ciò non nascondo una certa simpatia per questo rettile digitale (sì, lo so che il nome non deriva dal pitone, lo scriverò anche più avanti), perché il sottoscritto, come molti della Gen X, proviene dal mondo del BASIC V2 di Sua Maestà il Commodore 64. Per cui, la sopracitata modalità di dare vita alle variabili la prima volta che compaiono da qualche parte nel codice, ha una sua certa «reminiscenza».

E, no, non sono un Highlander. Per chi ha colto la citazione: la testa l'ho già persa e non è servito decapitarmi!

E forse è stata proprio questa la fortuna di questo linguaggio, o almeno una delle sue fortune: quella di attirare il bambino interiore di quei nerd cresciuti che siamo noi degli anni '70/'80 e di permetterci di tornare a giocare come una volta, ma creando stavolta librerie e framework decisamente più utili dei nostri giochini con cui stressavamo quel povero e lento interprete BASIC.

È allora con questo spirito giocoso che ho voluto buttare giù qualcosa che avesse un non so che di utile, mischiato a un non so che di leggero e di nerd.

Nel libro troverete subito il classico Hello World, naturalmente in salsa nerd e con tanto di pioggia digitale alla Matrix. Poi ci cimenteremo con un equalizzatore grafico e con diversi videogiochi: dal semplice T-Rex della modalità offline di Chrome al ben più articolato Rogue-like. Per i più matematici ci sarà anche il «Gioco della Vita», oltre a parecchia altra roba interessante.

Ma tutti i programmi sono accomunati da una caratteristica fondamentale: girano rigorosamente dentro la finestra del terminale.

Altrimenti, che libro «per nerd» sarebbe?!

> 🐍 **Versione di Python di riferimento**: Tutto il codice e i progetti di questo libro sono stati sviluppati e verificati con **Python 3.10 o versioni successive**, facendo largo uso delle annotazioni di tipo moderne (come le unioni native con l'operatore pipe `|` e le collezioni generiche tipo `list[int]`). Se sulla vostra macchina avete una versione più datata, questo è il momento perfetto per aggiornarla prima di cominciare!

Ehi, tu con Windows: ti vedo! 🪟

Ora, siamo onesti: un libro che si intitola *«Python per Nerds»* e che fa girare tutto su schermo nero a caratteri fosforescenti dà per scontato che stiate programmando su una fiammeggiante macchina Linux (o tutt'al più su macOS armati di terminale zsh e Homebrew). In quell'ambiente, la libreria `curses` fa parte della dotazione standard di Python fin dal Pleistocene e tutto funziona al primo colpo senza installare nulla.

Se però fate parte di quella frangia di eretici (o siete semplicemente costretti a usare il laptop aziendale sotto lo sguardo severo dell'amministratore di sistema) che programma su **Windows**, vi accorgerete subito di una cosa spiacevole: lanciando uno script che contiene `import curses`, Python vi risponderà con un antipatico `ModuleNotFoundError: No module named '_curses'`. Il motivo è storico e architetturale: la libreria Curses (`ncurses`) è nata per i terminali Unix e lo standard POSIX. Poiché Windows ha storicamente utilizzato un'architettura di console completamente diversa (le API Win32 Console), la distribuzione ufficiale di CPython per Windows (quella scaricabile da `python.org`) non include il modulo `curses` precompilato.

Niente panico, avete due strade per godervi il libro:

1. **La via maestra del vero Nerd: WSL (Windows Subsystem for Linux)**. È la soluzione di gran lunga migliore e raccomandata. Aprite PowerShell come amministratore, digitate `wsl --install`, riavviate e vi ritroverete una vera shell Ubuntu Linux dentro Windows. WSL rappresenta l'ambiente Windows più vicino a quello Unix nativo e offre il massimo livello di compatibilità con i programmi del libro (anche se la propagazione audio e certi font monospace di Windows Terminal potrebbero richiedere una piccola messa a punto locale).
2. **La scorciatoia pragmatica su Windows nativo**: se volete rimanere nel Prompt o in PowerShell, vi basta digitare una volta sola:

   ```
   pip install windows-curses
   ```

   È un pacchetto della community che ricrea l'interfaccia Curses su Windows. Con questo trucco, quasi tutti i progetti del libro (Matrix, Dino Run, Game of Life, Fireworks, PyRogue e la modalità 3D dell'ASCII Video) funzioneranno nativamente. Per i capitoli multimediali più avanzati (come l'equalizzatore che si interfaccia ai server audio Linux o il player video che aggancia i decoder di sistema), WSL rimane comunque la scelta d'elezione per non impazzire.

Ogni capitolo riguarda uno dei programmi citati e affronta determinati aspetti del linguaggio, in un crescendo di complessità man mano che si procede.

Si inizierà sempre con una breve introduzione, più o meno storica, sull'argomento, per poi entrare nel vivo delle peculiarità tecniche introdotte nel capitolo. Seguirà il commento delle righe più significative del programma in esame. Il tutto si concluderà con alcune proposte di esercizi a tema.

Il formato digitale del libro consente di tenerlo aperto mentre, dall'altra parte dello schermo, potete stressare Python digitando con le vostre dita cicciotte e unte su VS Code.

Perché usate VS Code, VERO??

Ah, vietato usare l'AI.

Anche perché l'ho esaurita io per scrivere i programmi. 😀

Lei non sa chi sono io



Francesco Clementoni
Qui dovrei presentarmi, ma tanto salterete (come farei anche io) il paragrafo, per cui potrei metterci dentro un Lorem-ipsum e nessuno se ne accorgerebbe, ahahaha.

> Ma ho seminato indizi nel titolo precedente, per cui mi limito a dire che sono Ingegnere Informatico del vecchio (e glorioso) ordinamento (IO C'ERO, GANDALF, QUANDO NACQUE JAVA!!) e che se ho intrapreso quegli studi e ho potuto approdare al lavoro dei miei sogni è grazie alla passione con cui fui attirato dal C64 nel fantastico mondo della programmazione.
>
> 👉 Profilo LinkedIn

Capitolo 0: Il riscaldamento da terminale (fondamentali rapidi di Python)

> *«Iniziare a contare da zero non è una stranezza dei programmatori, ma l'unico modo matematicamente elegante di calcolare la distanza da un'origine.»*
> — Tesi di **Edsger W. Dijkstra**, liberamente ispirata a *Why numbering should start at zero* (EWD831, 1982)

Se questo libro inizia dal **Capitolo 0** e non dal Capitolo 1, non è per vezzo tipografico: è perché chiunque abbia mai programmato sa che gli array iniziano rigorosamente dall'indice `0`. In informatica, un indice non indica «quale elemento» stai guardando in senso ordinale, ma **l'offset di memoria** rispetto all'inizio del blocco:

```
Indirizzo effettivo = Indirizzo_Base + (Indice * Dimensione_Elemento)
```

Al primo elemento corrisponde uno scostamento nullo (`offset = 0`). I linguaggi che iniziano gli indici da `1` (come Lua o il vecchio Fortran) adottano una convenzione puramente ordinale e antropomorfa, che allontana il codice dal modello fisico dell'hardware, costringendo sviluppatori e compilatori a calcoli di offset compensativi. Dijkstra lo spiegava con rigore matematico: l'indice 0 misura la distanza esatta dall'inizio del blocco. Noi siamo qui per fare cose serie, sporcarci le mani con il terminale e comprendere l'architettura della memoria, quindi partiamo rigorosamente da zero.

Un piccolo chiarimento prima di cominciare: **questo libro non è destinato a chi non ha mai visto una riga di codice in vita sua**. Non ti spiegheremo che cos'è un computer, come scaricare l'eseguibile di Python dal web o cos'è una cartella. Diamo per scontato che tu abbia già scritto qualche script, che tu conosca la differenza tra una stringa e un intero e che tu abbia già assaporato la frustrazione di un errore di sintassi alle due di notte.

Questo Capitolo 0 è una **sessione di riscaldamento ad alta intensità**: un ripasso dei meccanismi interni del linguaggio, con particolare attenzione a quelle sfumature architetturali (gestione della memoria, scope, mutabilità, orientamento agli oggetti, bitwise) che faranno la differenza tra un videogioco fluido a 60 FPS nel terminale e un programma pieno di glitch involontari.

Direi che è ora di sporcarci le mani. *It's time to kick ass and chew bubble gum… and I'm all outta gum.* (cit. Duke Nukem)

---

1. La nascita di un serpente che serpente non era

Alla fine del **1989**, nei laboratori del CWI (*Centrum Wiskunde & Informatica*) di Amsterdam, un programmatore olandese di nome **Guido van Rossum** si trovò di fronte a una settimana di ferie natalizie con gli uffici deserti. Molti avrebbero acceso il camino bevendo cioccolata calda; Guido, annoiato, decise di scrivere da zero l'interprete per un nuovo linguaggio di scripting.

Guido voleva uno strumento che colmasse il baratro tra il linguaggio **C** (potente ed efficiente per i sistemi operativi, ma verboso, punitivo e con la fastidiosa abitudine di mandare in crash la macchina al primo puntatore fuori posto) e le shell di script Unix come Bash (comode per concatenare comandi, ma lente, fragili e ingestibili su progetti complessi). Prese a modello il linguaggio didattico ABC, conservandone l'eleganza sintattica, e vi innestò la gestione delle eccezioni, i tipi di dato dinamici e una stretta integrazione con l'ambiente Unix.

E il nome? **Nessun serpente.**
Guido era un fanatico della serie comica britannica della BBC degli anni '70: il **Monty Python's Flying Circus**. Il nome del linguaggio nacque da quell'umorismo dissacrante e surreale (motivo per cui, nella documentazione ufficiale di Python, le variabili di esempio storiche non si chiamano quasi mai `foo` e `bar`, ma `spam`, `eggs` e `lumberjack`).

Per quasi trent'anni, la comunità ha riconosciuto a Guido il titolo onorifico di **BDFL** (*Benevolent Dictator For Life*, «Dittatore Benevolo a Vita»), fino alle sue dimissioni nel 2018 in favore di un comitato di guida collegiale (*Steering Council*). La filosofia del linguaggio, immortalata da Tim Peters nel celebre **Zen di Python** (che puoi leggere nel terminale digitando `python3 -c "import this"`), si riassume in pochi comandamenti cardine:

> - *Il bello è meglio del brutto.*
> - *L'esplicito è meglio dell'implicito.*
> - *Il semplice è meglio del complesso.*
> - *La leggibilità conta.*

---

2. Strutture di memoria: in Python tutto è un oggetto (e i puntatori ci sono)

Una delle bugie più diffuse tra chi studia Python a livello base è: *«Python è fantastico perché non ha i puntatori come il C»*.
La realtà ingegneristica è esattamente opposta: **in Python non c'è nient'altro che puntatori** (o, per usare il lessico formale del linguaggio, *etichette e binding a oggetti*, che nell'interprete di riferimento CPython sono implementati internamente proprio come puntatori C).

Nel modello concettuale dei linguaggi a basso livello come il C (immaginando la memoria fisica senza le ottimizzazioni aggressive del compilatore che terrebbero i valori direttamente nei registri della CPU), dichiarare una variabile locale alloca uno spazio fisico contiguo di byte direttamente nello Stack. Quando scrivi:

```c
int x = 42;
```

il compilatore associa direttamente l'identificatore simbolico `x` a uno specifico indirizzo di memoria dello Stack (ad esempio `0x7ffd5e20`). La sequenza di bit che codifica il numero 42 (su 4 byte a 32 bit: `0x0000002A`) viene scritta fisicamente all'interno di quella cella. In C, la variabile `x` coincide con la sua memoria: leggere o scrivere `x` significa accedere direttamente a quei 4 byte nello Stack, senza alcun puntatore o livello di indirezione intermedio.

In Python, invece, **le variabili non sono scatole che contengono dati: sono etichette simboliche (nomi) che puntano a oggetti allocati nella memoria dinamica (la Heap)**.

Ricordiamo infatti che l'interprete standard e ufficiale di Python (quello avviato dal comando `python3`) è **CPython**, un software scritto nel linguaggio C. Pertanto, quando creiamo o manipoliamo un qualsiasi dato in Python, sotto il cofano CPython alloca nella memoria RAM una `struct` C (il costrutto nativo del linguaggio C utilizzato per aggregare più campi contigui in un unico blocco di memoria).

Ogni volta che scrivi una semplice assegnazione:

```python
punteggio = 42
```

l'interprete CPython non scrive il numero 42 direttamente in una cella di memoria: compie due operazioni distinte. Alloca (o recupera) un oggetto sulla memoria Heap e associa l'etichetta `punteggio` a quell'indirizzo.

Anatomia di un intero: la struct `PyLongObject`

A differenza dei 4 byte grezzi del C, in Python non esistono valori numerici «nudi». Ogni valore è incapsulato in una `struct` C che su architetture a 64 bit occupa 28 byte:

- **`ob_refcnt`** (8 byte): contatore di riferimenti (*Reference Counting*) per la gestione automatica della memoria;
- **`ob_type`** (8 byte): puntatore al tipo `PyLong_Type`, contenente la tabella delle operazioni aritmetiche;
- **`ob_size`** (8 byte): memorizza il segno e il numero di blocchi per supportare interi a precisione arbitraria senza rischio di overflow;

- **`ob_digit`** (4 byte): l'array che memorizza il valore numerico effettivo.

Possiamo verificare direttamente questo ingombro nel terminale interattivo tramite il modulo `sys`:

```
>>> import sys
>>> sys.getsizeof(42)
28
```

I 24 byte di metadati aggiuntivi sono il prezzo da pagare per il dinamismo e la sicurezza offerti dal linguaggio.

Nomi, binding e l'indirizzo di memoria (`id`)

Una volta creato l'oggetto sulla Heap, CPython aggiorna lo spazio dei nomi locale (*Namespace*): registra la stringa `"punteggio"` nel dizionario delle variabili locali e vi associa il puntatore a 64 bit che fa riferimento alla struct appena allocata.

In Python le variabili sono puri puntatori simbolici. Possiamo verificare l'indirizzo di memoria effettivo assegnato da CPython tramite la funzione built-in `id()`:

```
a = [1, 2, 3]
b = a
print(id(a) == id(b))  # Stampa True: puntano allo stesso identico indirizzo di memoria!
```

La cache dei piccoli interi e la trappola di `is`

Allocare continuamente 28 byte sulla Heap per ogni contatore `i` in un ciclo `for` frammenterebbe rapidamente la memoria. Per questa ragione CPython adotta un'ottimizzazione pragmatica: all'avvio del processo prealloca un array globale contenente tutti gli interi compresi tra `-5` e `256` (l'array `small_ints`).

Quando nel codice compare un numero in questo intervallo, Python non alloca nuova memoria: restituisce semplicemente il puntatore all'oggetto già esistente, condividendolo tra tutte le variabili che ne fanno richiesta:

```
# Entro la cache [-5, 256]: stesso identico oggetto in memoria
>>> a = 42
>>> b = 42
>>> a is b
True

# Fuori dalla cache: due oggetti distinti sulla Heap
>>> x = 1000
>>> y = 1000
>>> x is y
False
```

Questo comportamento svela una distinzione fondamentale tra i due operatori di confronto:

- **`==`** confronta l'**uguaglianza di valore** (`x == y` verifica se i dati sono equivalenti);
- **`is`** confronta l'**identità dell'oggetto** in memoria (`x is y` verifica se `id(x) == id(y)`).

Non usare mai `is` per confrontare numeri o stringhe: il fatto che funzioni con i piccoli interi è un semplice dettaglio implementativo di CPython su cui non bisogna mai fare affidamento. Nel codice idiomatico, `is` si usa quasi esclusivamente per verificare valori sentinella come `if x is None:`.

Mutabili vs immutabili: la frontiera dei bug silenziosi

Questa distinzione è il terreno su cui inciampano molti programmatori quando si addentrano un po' oltre al rassicurante «Hello World»:

- **Tipi Immutabili:** `int`, `float`, `str`, `tuple`, `bool`, `bytes`, `frozenset`.
  Una volta creati nella memoria Heap, il loro contenuto **non può mai essere modificato**. Se scrivi `x = 1000` e poi `x += 1`, non stai modificando il numero 1000: stai allocando un *nuovo* oggetto `1001` in un altro indirizzo di memoria e stai spostando l'etichetta `x` su di esso.
- **Tipi Mutabili:** `list`, `dict`, `set`, array di byte, istanze di classi ordinarie.
  Il loro contenuto interno può essere alterato direttamente sul posto (*in-place*) **mantenendo lo stesso identico indirizzo di memoria**.

Il nesso fatale: puntatori condivisi + modifica in-place
Perché questa distinzione genera bug devastanti se combinata con l'operatore di assegnazione `=`?
Ricorda che in Python l'operatore `=` **non duplica mai i dati**, ma si limita a collegare una nuova etichetta (un alias/puntatore) allo stesso identico oggetto in memoria.

Se l'oggetto è **mutabile**, ecco cosa accade dietro le quinte:

```
# 1. Creiamo una lista in memoria (es. all'indirizzo Heap 0x1A4F)
giocatore_a = [10, 20] # Coordinate [y, x]

# 2. L'assegnazione crea solo un SECONDO puntatore allo STESSO indirizzo 0x1A4F!
giocatore_b = giocatore_a

# 3. Essendo la lista MUTABILE, modifichiamo il dato sul posto:
#    l'oggetto all'indirizzo 0x1A4F viene alterato internamente SENZA spostare il
puntatore!
giocatore_b[0] = 99

# 4. 'giocatore_a' punta ancora al medesimo indirizzo 0x1A4F, che è appena stato alterato:
print(giocatore_a) # Stampa [99, 20]! Hai modificato involontariamente anche il Giocatore
A!
```

Guarda invece cosa accadrebbe con un tipo **immutabile** (come un intero o una tupla):

```
punteggio_a = 1000
punteggio_b = punteggio_a  # Entrambi puntano all'oggetto intero 1000

# Gli interi NON possono essere mutati sul posto!
# Python non può alterare il 1000: alloca un nuovo oggetto 1500 a un nuovo indirizzo e
riassegna 'punteggio_b'
punteggio_b += 500

print(punteggio_a) # Stampa 1000: intatto e protetto!
print(punteggio_b) # Stampa 1500
```

L'effetto a catena (`giocatore_a` che cambia a sorpresa) si verifica quindi **solo e soltanto perché** la mutabilità consente di alterare il contenuto dell'indirizzo condiviso senza creare un nuovo oggetto.

Per duplicare un contenitore mutabile senza condividere la memoria devi usare una copia esplicita:

- Copia superficiale (*shallow copy*): `giocatore_b = giocatore_a.copy()` oppure con lo slicing `giocatore_b = giocatore_a[:]`;
- Copia profonda per strutture annidate complesse (*deep copy*): `import copy; copia = copy.deepcopy(matrice)`.

La celeberrima trappola del parametro di default mutabile

Guarda questa funzione apparentemente innocua:

```python
def spawna_nemico(tipo: str, poteri: list[str] = []) -> dict:
    poteri.append("morso")
    return {"tipo": tipo, "poteri": poteri}

orco = spawna_nemico("Orco")
goblin = spawna_nemico("Goblin")
print(goblin["poteri"]) # Stampa ['morso', 'morso']! Perché?!
```

In Python, le definizioni delle funzioni vengono eseguite **una sola volta** al momento del caricamento del modulo, non ogni volta che la funzione viene invocata. L'argomento di default `poteri=[]` viene istanziato all'avvio e condiviso tra tutte le chiamate successive!
La soluzione corretta, consolidata e idiomatica raccomandata dalla comunità Python come *best practice* irrinunciabile è usare sempre l'immutabile **`None` come valore sentinella**:

```python
def spawna_nemico(tipo: str, poteri: list[str] | None = None) -> dict:
    if poteri is None:
        poteri = []
    poteri.append("morso")
    return {"tipo": tipo, "poteri": poteri}
```

Perché usare `None` e non limitarsi a verificare se la lista sia vuota (ad esempio con `if len(poteri) == 0: poteri = []`)?
Perché in Python `None` distingue in modo pulito **l'assenza di un argomento** dall'invio intenzionale di un contenitore:

- Se il chiamante **non passa nulla**, `poteri` è `None`: la funzione crea una nuova lista locale isolata;
- Se il chiamante **passa una propria lista vuota** (es. `inventario = []`), `poteri` **non è `None`**: la funzione lavorerà direttamente sulla lista fornita dal chiamante, popolandola correttamente sul posto senza sostituirla con una nuova variabile locale.

---

3. Liste, tuple, slicing e matrici 2D da combattimento

Nei nostri progetti terminal-based faremo un uso massiccio di collezioni ordinate per rappresentare particelle, code di caratteri, mappe di gioco ed equalizzatori spettrali.

Le tuple per le coordinate e l'unpacking

Le tuple `(y, x)` sono sequenze immutabili, occupano meno memoria delle liste e possono essere usate come chiavi nei dizionari o negli insiemi (`set`). Inoltre permettono l'**Unpacking** (spacchettamento) immediato:

```python
coordinate = (12, 45)
riga, colonna = coordinate  # Unpacking pulito in due variabili distinte

# Spacchettamento pratico durante l'iterazione di una lista di entità:
posizioni_nemici = [(5, 10), (12, 25), (18, 40)]
for y, x in posizioni_nemici:
    pass  # Elabora direttamente ciascuna coordinata y e x
```

Slicing esteso: `[start:stop:step]`

L'operatore di slicing non serve solo a prendere fette di stringhe o liste, ma supporta un terzo parametro fondamentale, il passo (*step*):

```
alfabeto = "abcdef"
print(alfabeto[::2])   # Stampa 'ace' (salta di 2 in 2)
print(alfabeto[::-1])  # Stampa 'fedcba' (inverte la sequenza istantaneamente!)
```

Nel Capitolo 1 useremo proprio questo trucco per invertire la stringa del messaggio segreto di Matrix prima di farla scorrere verticalmente verso il basso.

List comprehension

Invece di creare una lista vuota e popolarla con un ciclo `for` di 4 righe, Python permette una sintassi dichiarativa, compatta ed efficiente (che a livello di bytecode ottimizza l'allocazione interna evitando il continuo lookup dinamico del metodo `.append()`):

```
# Modo prolisso
particelle_attive = []
for p in lista_particelle:
    if p.in_vita:
        particelle_attive.append(p)

# Modo Pythonico e compatto (List Comprehension)
particelle_attive = [p for p in lista_particelle if p.in_vita]
```

Come costruire una matrice 2D (senza cadere nel burrone)

Nel Capitolo 3 (Game of Life) e nel Capitolo 7 (PyRogue) lavoreremo con griglie bidimensionali (righe per colonne). C'è un errore fatidico che commette chiunque almeno una volta nella vita:

```
# MODO ERRATO: una trappola classica dei riferimenti condivisi
griglia_sbagliata = [[0] * 5] * 5
griglia_sbagliata[0][0] = 99
# Risultato: TUTTE le 5 righe avranno il 99 nella prima colonna!
# Perché l'operatore * ha copiato 5 volte il PUNTATORE alla stessa identica lista interna!

# MODO CORRETTO: Creare una nuova lista indipendente per ogni riga
ALTEZZA = 5
LARGHEZZA = 5
griglia_corretta = [[0 for _ in range(LARGHEZZA)] for _ in range(ALTEZZA)]
griglia_corretta[0][0] = 99 # Modifica ESCLUSIVAMENTE la cella (0, 0)!
```

---

4. Indentazione e sintassi: la regola off-side

Nel 1966, l'informatico britannico Peter Landin teorizzò la *Off-side Rule*: l'idea che la struttura a blocchi di un linguaggio di programmazione potesse essere espressa direttamente dall'indentazione visiva del codice, anziché da delimitatori espliciti come parentesi graffe `{}` o parole chiave `begin/end`.

Guido van Rossum scelse di fare dell'indentazione una parte formale della grammatica di Python:

- **Niente graffe:** Il codice è naturalmente compatto e privo di rumore visivo;
- **Leggibilità forzata:** Non puoi scrivere codice disallineato e caotico, perché l'interprete si rifiuterà di compilarlo sollevando un `IndentationError`;
- **La regola aurea (PEP 8):** Usare rigorosamente **4 spazi per livello**. Mai usare tabulazioni o, peggio ancora, mescolare spazi e tabulazioni (un tab su un editor può valere 8 caratteri e su un altro 4, generando discrepanze invisibili all'occhio umano ma catastrofiche per il parser).

5. Controlli di flusso, cicli e iteratori

Condizioni ed EAFP

Oltre ai costrutti tradizionali `if`, `elif`, `else` e all'operatore ternario compatto (`valore_se_vero if condizione else valore_se_falso`), in Python vige la filosofia **EAFP** (*Easier to Ask for Forgiveness than Permission*, «È più facile chiedere perdono che permesso»).

Nei linguaggi tradizionali spesso si controlla preventivamente ogni condizione prima di accedere a una risorsa (*LBYL*, *Look Before You Leap*):

```
# Approccio LBYL (Guardingo)
if x in inventario:
    usa(inventario[x])
```

In puro stile pythonico, quando ha senso architetturale, si preferisce tentare l'azione e catturare l'eccezione mirata con un blocco `try/except`:

```
# Approccio EAFP (Pythonico)
try:
    usa(inventario[x])
except KeyError:
    pass # L'oggetto non era presente, gestisci senza crash
```

I migliori alleati nel terminale: `enumerate()` e `zip()`

In linguaggi a basso livello come il C, per iterare su un array si usa tradizionalmente un contatore numerico esplicito:

```
// Stile C classico: contatore manuale per accedere all'indice
for (int i = 0; i < lunghezza; i++) {
    elabora(array[i]);
}
```

Chi proviene da questo background tende spesso a trasporre meccanicamente lo stesso schema in Python scrivendo `for i in range(len(lista)):` e accedendo poi a `lista[i]`. In Python questo costrutto è ridondante, poco leggibile e non idiomatico.

Quando ci serve sia l'indice numerico sia l'oggetto (ad esempio per calcolare la riga Y su cui disegnare uno sprite a più linee in Curses), Python mette a disposizione la funzione built-in **`enumerate()`**:

- **Cosa fa `enumerate()`**: riceve un iterabile e restituisce un iteratore di **tuple a due elementi** composte da `(indice, elemento)` (con indice che parte da `0` per default);
- **Perché il ciclo ha due variabili di scorrimento**: sfrutta l'**unpacking automatico delle tuple** (che abbiamo visto nella Sezione 3). A ogni giro del ciclo, la coppia restituita viene spacchettata simultaneamente nella variabile contatore e nella variabile che ospita il dato:

```
sprite = ["  O  ", " /|\\ ", " / \\ "]

# Coordinate di partenza a schermo (riga Y, colonna X)
pos_y = 10
pos_x = 20
```

```
# A ogni ciclo, enumerate produce (0, "  O  "), poi (1, " /|\\ "), ecc.
# 'offset_y' cattura l'indice di riga (0, 1, 2) e 'riga_testo' il contenuto ASCII:
for offset_y, riga_testo in enumerate(sprite):
    # La funzione addstr(y, x, testo) di Curses disegna ogni riga scendendo lungo l'asse Y
    stdscr.addstr(pos_y + offset_y, pos_x, riga_testo)
```

E con `zip()` possiamo scorrere in parallelo due o più sequenze contemporaneamente:

```
altezze_barre = [12, 18, 5, 22]
colori = [1, 2, 3, 2]
for altezza, colore in zip(altezze_barre, colori):
    disegna_barra(altezza, colore)
```

---

6. Scope dei nomi e namespace: la regola LEGB

Uno degli aspetti che genera maggiore confusione nei neofiti è capire **come vengono risolti gli identificatori** (ovvero i nomi associati a variabili, funzioni, classi o moduli) e a quale oggetto facciano riferimento all'interno di funzioni nidificate o metodi di classe.

In Python, infatti, anche le funzioni e i tipi sono oggetti di prima classe (*first-class objects*): il nome `print` o `len` non è una parola chiave speciale del compilatore, ma un comune identificatore registrato in un namespace che punta a un oggetto eseguibile.

Quando nel codice richiami un qualsiasi nome, l'interprete lo cerca rigorosamente attraverso quattro livelli gerarchici sequenziali, sintetizzati dall'acronimo **LEGB**:

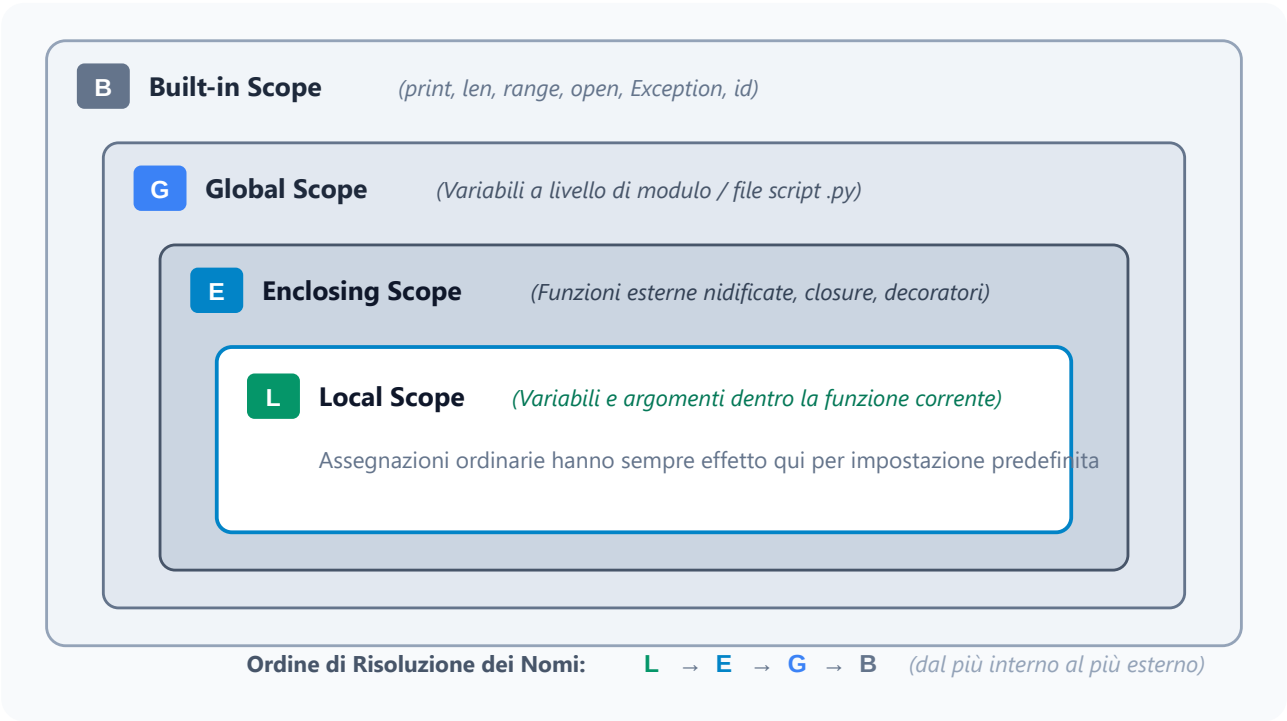


Figura 1: Gerarchia di Risoluzione dello Scope LEGB in Python

1. **Local (L):** Nomi (variabili e funzioni locali) creati o assegnati all'interno del corpo della funzione corrente;
2. **Enclosing (E):** Nomi definiti nel corpo di funzioni esterne o genitrici (tipico delle *closure* e delle funzioni nidificate);

3. **Global (G):** Nomi definiti a livello di modulo (variabili a livello di file, funzioni `def` e classi `class` del sorgente corrente);
4. **Built-in (B):** Nomi predefiniti forniti automaticamente all'avvio dell'interprete (`print`, `len`, `range`, `int`, `Exception`, `id`, ecc.).

Proprio perché le funzioni standard risiedono nel livello più esterno (**Built-in**), definire un nome identico a livello locale lo «oscura» (*shadowing*) immediatamente:

```
def esempio_shadowing():
    len = 10  # Creiamo un nome locale 'len' (Local vince su Built-in!)
    # len([1, 2, 3])  <-- ERRORE! TypeError: 'int' object is not callable!
```

La trappola della modifica e la parola chiave `global`

In Python, la sola **lettura** di una variabile globale all'interno di una funzione funziona senza problemi risalendo la gerarchia LEGB:

```
punteggio = 100  # Variabile a livello globale (G)

def mostra_punteggio():
    print(punteggio)  # OK: non trovandola in Local (L), Python sale a Global (G) e stampa
100
```

Il problema si manifesta nel momento in cui provi a **modificarla**:

```
punteggio = 100  # Globale (G)

def incrementa():
    punteggio = punteggio + 10  # <-- ERRORE! UnboundLocalError!
```

Perché si verifica questo errore?
Quando Python compila il corpo della funzione, nota la presenza dell'assegnazione `punteggio = ...` e classifica istantaneamente `punteggio` come **variabile locale (L)** per l'intera funzione.
Quando poi esegue la riga, deve prima valutare il lato destro dell'uguale (`punteggio + 10`); cercando `punteggio` nello scope locale, scopre che a livello locale non gli è stato ancora assegnato alcun valore prima di questa riga. Non potendo risalire al globale (avendo già marcato la variabile come locale), l'interprete lancia l'errore `UnboundLocalError`.

Per istruire Python a non creare una variabile locale, ma a modificare quella globale esistente a livello di modulo, si usa la parola chiave `global`:

```
punteggio = 100

def incrementa():
    global punteggio  # Istruisce Python a fare riferimento alla variabile globale del
modulo
    punteggio += 10   # Ora modifica correttamente il valore globale a 110!
```

> 📌 **Nota:** Nel codice professionale e nei videogiochi complessi, l'abuso della parola chiave `global` è considerato un anti-pattern architetturale. Rende difficile prevedere chi modifica cosa e genera bug subdoli. Nei capitoli avanzati vedremo come incapsulare lo stato del gioco dentro classi dedicate (come `Engine` o `SpettacoloPirotecnico`), dove i dati risiedono comodamente come attributi d'istanza (`self.punteggio`).

7. La programmazione a oggetti (OOP): un'anima pratica costruita sui dizionari

Sebbene il costrutto `class` fosse presente fin dai primissimi rilasci pubblici del linguaggio (versione 0.9.0 nel febbraio 1991), Python è nato con un'impostazione concettuale profondamente procedurale e modulare. A differenza di linguaggi monolitici come C++ o Java, dove la gerarchia delle classi è un dogma imposto fin dal `main()`, in Python la Programmazione a Oggetti è stata concepita con un'impronta pragmatica, dinamica e priva di cerimoniali burocratici (la cui completa unificazione tra tipi nativi e classi definite dall'utente, le cosiddette *new-style classes*, è giunta con Python 2.2 e 3.0).

Dietro le quinte: gli attributi delle istanze e il loro `__dict__`

In linguaggi compilati rigidi come il C++, la struttura in byte di un oggetto è scolpita nella pietra al momento della compilazione (*vtable* e memoria statica).
In Python, nel caso ordinario, gli attributi di un'istanza risiedono all'interno di un comune dizionario hash accessibile tramite `__dict__` (sebbene esista la direttiva speciale `__slots__` per eliminare il dizionario e compattare la memoria in array statici per simulazioni ad alte prestazioni con decine di migliaia di particelle):

```
class Particella:
    def __init__(self, x: float, y: float):
        self.x = x
        self.y = y

p = Particella(10.0, 5.0)
print(p.__dict__) # Stampa: {'x': 10.0, 'y': 5.0}
```

Questo rende Python dinamico e riflessivo: puoi ispezionare, aggiungere o rimuovere attributi a runtime con estrema naturalezza.

Il mistero di `self`: perché dobbiamo scriverlo (quasi) sempre?

A chi approda a Python da linguaggi come C++, Java o C#, l'uso pervasivo di `self` appare spesso come una bizzarria ridondante. Ci si pongono due domande legittime:

1. *Perché devo dichiararlo come primo parametro nei metodi (`def metodo(self, ...)`), se poi quando chiamo `oggetto.metodo(...)` non lo passo?*
2. *Perché dentro il corpo del metodo devo scrivere sempre `self.x` per accedere a un campo, invece di scrivere direttamente `x` come in Java?*

Analizziamo i motivi ingegneristici di queste due scelte:

1. Definizione esplicita vs invocazione automatica (*bound methods*)
In Python, una classe è prima di tutto un contenitore di funzioni. Quando dichiari un metodo d'istanza:

```
class Particella:
    def __init__(self, x: float, y: float):
        self.x = x
        self.y = y

    def sposta(self, dx: float, dy: float):
        self.x += dx
        self.y += dy
```

La funzione `sposta` è memorizzata nella classe e richiede 3 parametri: `(self, dx, dy)`.
Quando invochi il metodo attraverso un'istanza:

```
p = Particella(10.0, 20.0)

# 1. Chiamata comune ad alto livello (passi solo 2 argomenti):
p.sposta(1.5, -2.0)

# 2. Cosa esegue REALMENTE l'interprete sotto il cofano:
Particella.sposta(p, 1.5, -2.0)
```

Concettualmente, l'invocazione tramite sintassi a punto `p.sposta(...)` equivale a richiamare la funzione direttamente dalla classe (`Particella.sposta(p, ...)`), passando l'istanza `p` come **primo argomento effettivo**. Il parametro formale `self` (abbreviazione convenzionale per *«me stesso»*) è la variabile locale che riceve il riferimento a quell'istanza.

2. Niente `this` implicito: *«Explicit is better than implicit»*
Nei linguaggi con il puntatore `this` implicito (come C++ o Java), dentro un metodo puoi scrivere `x = 5;` e il compilatore indovina se `x` è una variabile locale o un attributo dell'oggetto. Nelle classi con molti campi, questo crea ambiguità visiva.

In Python, la regola LEGB non fa sconti:

- Se scrivi `x = 5`, crei una variabile **strettamente locale** che viene distrutta al termine del metodo;
- Se vuoi salvare il dato nell'oggetto che galleggia sulla Heap, **devi** scrivere esplicitamente `self.x = 5`.

L'uso esplicito di `self.` rende trasparente a colpo d'occhio l'intenzione del programmatore: ogni volta che vedi `self.`, sai con certezza che stai alterando l'attributo persistente di quell'istanza (memorizzato nel suo `__dict__`) e non una variabile temporanea.

> **Curiosità da Nerd: `self` è una parola chiave?**
> Sorprendentemente, `self` **non è una parola riservata** di Python. Potresti chiamare quel primo parametro `me`, `istanza` o perfino `this`, e il codice funzionerebbe esattamente nello stesso modo. Tuttavia, la convenzione di chiamarlo `self` è uno standard universale così sacro (sancito dalla PEP 8) che violarlo è considerato un gravissimo errore di stile.
> L'unica eccezione in cui `self` non si usa è nei metodi che non operano sulla singola istanza: i metodi statici decorati con `@staticmethod` (che non ricevono argomenti impliciti) e i metodi di classe `@classmethod` (dove il primo parametro riceve la classe stessa ed è convenzionalmente chiamato `cls`).

«We Are All Consenting Adults Here»

In Python non esistono parole chiave come `private`, `protected` o `public`. Non puoi impedire fisicamente a un altro modulo di accedere a una variabile interna di una classe.

La filosofia del linguaggio si basa sul rispetto reciproco tra programmatori (*«Siamo tutti adulti consenzienti»*):

- Se un attributo o metodo è pensato per uso interno e non fa parte dell'interfaccia pubblica, lo si precede con un **singolo trattino basso** (es. `_genera_particella()`);
- Se vuoi evitare che una sottoclasse sovrascriva accidentalmente un attributo interno, usi il **doppio trattino basso** (es. `__id_univoco`), che attiva il meccanismo di *Name Mangling* (l'attributo viene internamente rinominato in `_NomeClasse__id_univoco`).

Decoratori moderni: `@property` e `@dataclass`

Nei capitoli successivi sfrutteremo due costrutti moderni della libreria standard che rendono l'OOP pulita e priva di codice ripetitivo (*boilerplate*):

1. **Il decoratore `@property`:** ti consente di esporre un metodo come se fosse un normale attributo di sola lettura, calcolandone il valore dinamicamente:

```python
class Razzo:
    def __init__(self, vita_massima: int):
        self.vita = vita_massima

    @property
    def e_spento(self) -> bool:
        return self.vita <= 0

r = Razzo(10)
if r.e_spento: # Niente parentesi tonde: sintassi fluida e pulita!
    pass
```

2. **Le `@dataclass` (dal modulo `dataclasses`):** eliminano decine di righe di codice ripetitivo (*boilerplate*), generando automaticamente `__init__`, una rappresentazione leggibile `__repr__` e il controllo di uguaglianza strutturale `__eq__`:

```python
from dataclasses import dataclass

@dataclass
class Punto:
    x: float
    y: float
    simbolo: str = "*"

# 1. __init__ automatico: non serve scriverlo a mano, accetta parametri e default!
p1 = Punto(10.0, 20.0)

# 2. __repr__ automatico: utilissimo per il debug delle coordinate a terminale
print(p1)
# Output leggibile: Punto(x=10.0, y=20.0, simbolo='*')
# (In una classe ordinaria stamperebbe l'indirizzo grezzo: <__main__.Punto object at
0x7f...>)

# 3. Uguaglianza strutturale (__eq__):
p2 = Punto(10.0, 20.0)
print(p1 == p2)
# Output: True! Confronta il contenuto dei campi (x, y, simbolo).
# (In una classe ordinaria p1 == p2 restituirebbe False perché confronterebbe i
puntatori!)
```

---

8. I segreti del terminale: Curses, bitwise e aspetto anamorfico

Per concludere questo riscaldamento, ecco tre regole fondamentali che governeranno ogni singola riga di codice grafico presente in questo libro:

1. La maledizione delle coordinate: è sempre `(y, x)`!

In matematica cartesiana e nella maggior parte delle librerie grafiche per finestre (come Pygame, OpenGL o SDL), una posizione è espressa come `(x, y)`: prima la larghezza orizzontale, poi l'altezza verticale.

Nel terminale e nella libreria Curses è l'esatto contrario: si indica prima la riga e poi la colonna, ossia `(y, x)`:

```
# Sintassi corretta per Curses: prima la quota Y (riga), poi la X (colonna)
stdscr.addch(y, x, '@')
```

Scambiare `(y, x)` con `(x, y)` è il bug numero uno in cui inciampano anche programmatori esperti. Quando il programma va in crash sollevando un'eccezione `curses.error: addch() returned ERR`, nel 99% dei casi hai tentato di scrivere a coordinata `X` come se fosse `Y`, finendo fuori dall'altezza del monitor!

2. Maschere bitwise per gli attributi grafici

Curses memorizza colore, brillantezza e stili visivi combinando numeri interi mediante **maschere di bit**.
Per applicare contemporaneamente un colore e il grassetto luminoso, si usa l'operatore **OR bit a bit (`|`)**:

```
# 1. Registra prima la coppia n. 1 (es. testo verde su sfondo nero)
curses.init_pair(1, curses.COLOR_GREEN, curses.COLOR_BLACK)

# 2. Combina l'ID della coppia registrata (1) con l'attributo luminoso (A_BOLD)
attributo = curses.color_pair(1) | curses.A_BOLD
stdscr.addstr(riga, colonna, "TESTO LUMINOSO", attributo)
```

3. La proporzione anamorfica 2:1 del terminale

Sullo schermo di un computer grafico, un pixel è un quadrato perfetto con rapporto 1:1.
In un emulatore di terminale a caratteri, invece, **una cella di testo è tipicamente rettangolare: alta circa il doppio di quanto sia larga (rapporto 2:1)**.

Se nel nostro codice simuliamo un'esplosione circolare o il rimbalzo di una pallina assegnando la stessa velocità orizzontale e verticale (`vx = 1.0`, `vy = 1.0`), sullo schermo non vedremo una sfera, ma un uovo allungato verso il basso!
Per ottenere forme geometriche simmetriche e cerchi perfetti, dovremo raddoppiare matematicamente le velocità o i raggi orizzontali:

```
offset_x = math.cos(angolo) * raggio * 2.0  # Compensazione del rapporto 2:1
offset_y = math.sin(angolo) * raggio
```

🎯 Si va in scena

Ora che abbiamo rispolverato il motore interno di Python, chiarito il modello di memoria e compreso le particolarità della griglia a caratteri, la teoria può lasciare spazio al codice vivo.

Nel prossimo capitolo abbandoneremo il rassicurante e noioso `print()` sequenziale per tuffarci nel cuore pulsante della libreria Curses: **è il momento di costruire la leggendaria Pioggia Digitale di Matrix**.

🚀 IL VIAGGIO NEL TERMINALE CONTINUA NEL LIBRO COMPLETO!

Ti aspettano **7 progetti completi** in puro Python: **Matrix Rain • Dino Run • Game of Life • Fireworks OOP • Equalizzatore FFT • ASCII Video & 3D Donut • PyRogue**.

Ti è piaciuto questo estratto? Acquista il libro completo su Amazon:

https://amzn.eu/d/0eBQKhIA